Verix PRD and Build Plan

Nov 3, 2026 · @Zhilo.labs

Overview

Verix v0 is a local-first tool that records what an AI coding agent read and assumed, then tells reviewers which of those assumptions no longer match the current code. It is built mostly from deterministic parts (Git, hashes, graph traversal, rules), with an LLM used only to extract and explain, never to decide.

Short answers to your questions

- **Is there an LLM that checks?** Yes, but only in a supporting role: it extracts claims from agent text, reads ambiguous diffs, and writes explanations. Whether an action is allowed is decided by deterministic rules. An LLM judging its own kind's output is the weakest link, so it never holds the gate.
- **Tech stack, in one line:** Python for the engine and CLI, SQLite locally and PostgreSQL in the cloud, Tree-sitter for code structure, FastAPI for the API, a GitHub App for PRs, and Next.js only after the benchmark passes. Each choice and the rejected alternatives are in the tech stack section.
- **How it gets built:** benchmark first, then local engine and Claude Code hook, then the GitHub App, then hosted features. The plan is in the build plan section.

Product goal. Make an agent's pull request show which of its assumptions went stale while it worked.

Goals

- G1: detect stale assumptions with at least 80% recall on seeded scenarios.
- G2: keep false blocks under 10% of valid actions.
- G3: add under 5 seconds of latency to a typical PR check (target to validate).
- G4: install in under 10 minutes without sending source code to a server.

Non-goals for v0 and v1

- Not a general memory store or vector database.
- Not an agent runtime or a replacement for Claude Code, Cursor or LangGraph.
- Not a security boundary or proof of code correctness.
- No automatic extraction of every dependency; no support for non-code domains yet.

Product success metrics (to validate, not yet measured): flags acted on by reviewers, PRs reworked after a flag, weekly active repos, and design partners who pay.

Evidence and strategy context is in the first document, *Verix: Idea, Problem and Fix*. Technology choices below are my engineering recommendations and have not been benchmarked.

Users and use cases

The primary user is an engineer running several coding agents against one repository. The primary beneficiary is the reviewer who must trust their pull requests. All personas below are hypotheses to test in interviews.

Persona	Context	Job to be done	Success looks like
Agent operator (primary)	Runs 3 or more Claude Code or similar agents in parallel or in the background	Know which agent PRs rest on outdated assumptions before merging	Fewer reworked PRs; clear re-check list per PR
Reviewer	Reviews agent-written PRs on a shared repo	Decide quickly whether the PR's premises still hold	Flags are specific and mostly correct
Agent-tool builder	Founder of a small coding-agent startup (3 to 30 engineers)	Add change-aware safety to their agent without building it	Integration in under a day through the SDK
Engineering lead	Owns merge policy for the repo	Set rules for when stale PRs are blocked	Policy file, audit trail, low false-block rate

Key scenarios

1. **Stale snapshot PR.** An agent starts at 10:00, another change lands at 10:45, the agent opens a PR at 11:30. The PR gets a comment listing the changed files it relied on and the claims now *suspect*.
2. **Resumed session.** An agent resumes after context compaction or a day away. Before its first protected edit, the hook says which remembered claims are stale and which checks refresh them.
3. **Two agents, one repo.** Agent B is about to edit a module that agent A just changed. The hook holds B's edit until B re-reads the current file.
4. **Valid change, no noise.** A PR touches files unrelated to anything the agent read. Verix stays silent and does not block.
5. **Unknown dependency.** The agent made a high-risk edit but its read log is thin. The result is *unresolved*, with a request for human review, never a false all-clear.

Out of scope users for v0: non-coding agent builders, enterprise compliance teams, and anyone needing legal or safety guarantees.

Functional requirements

Priority: P0 ships in v0, P1 in v1, P2 later. Each requirement has a testable acceptance criterion.

ID	Requirement	Pri	Acceptance criterion
F1	Capture reads: log every file the agent reads with repo, commit, path and SHA-256 of content	P0	For a scripted session, 100% of Read tool calls appear in the log with correct hashes
F2	Capture writes and proposed edits with target paths	P0	Every Edit/Write call is logged before it executes
F3	Register a claim with text, supporting evidence (paths, commit) and optional parent claims	P0	A claim cannot be created without at least one evidence reference, or it is stored as *unknown*
F4	Detect changes between two commits by path and content hash, ignoring whitespace-only changes when configured	P0	Rename, delete, modify and mode-change detected on a 20-case fixture
F5	Mark claims *suspect* when any evidence file changed	P0	Fixture: changing a cited file flips its claims to suspect within one analysis run
F6	Propagate suspicion through dependency edges to decisions and planned edits	P0	3-level chain fixture: all downstream nodes flagged; unrelated nodes untouched
F7	Honor independent evidence: a claim with a still-valid independent source is not escalated	P1	Fixture with two sources, one changed: claim stays valid with a note
F8	Evaluate a proposed action and return allow, verify or block with a reason and resolving checks	P0	Same inputs always give the same output; reasons name specific files and commits
F9	Verification planner suggests the cheapest resolving check (re-read, diff, test, human)	P1	Each suggested check, when run, resolves the flagged claim in 90% of fixtures

ID	Requirement	Pri	Acceptance criterion
F10	Claude Code hook: block protected edits when decision is block; surface checks to the agent when verify	P0	PreToolUse hook exits so the edit does not run; the agent sees the reason
F11	Revision-bound approvals: an allow expires if base revision or evidence changes before execution	P0	Race fixture: change between check and merge reopens the decision
F12	Policy file in repo (`.verix.yml`): protected paths, risk levels, advisory vs blocking mode	P0	Invalid policy is rejected with a line-level error
F13	GitHub App posts one PR comment and updates it on new commits	P1	One comment per PR, updated, not duplicated
F14	GitHub status check set to pending, success or failure by policy	P1	Branch protection can require it; advisory mode never fails a PR
F15	Local report (CLI and JSON) of claims, states, flags and decisions	P0	`verix report --json` validates against a published schema
F16	LLM claim extraction from agent notes, always labeled as extracted and validated against evidence	P1	Extracted claims without matching evidence are stored as unknown
F17	Audit log of every decision with policy version and inputs	P0	A decision can be replayed from the log with an identical result
F18	Benchmark harness: seed repo changes mid-task and score outputs	P0	One command runs 30 scenarios and prints recall, precision, false-block rate
F19	Hosted dashboard with an evidence chain per flag	P2	Click a flag, see change, claim, decision, action and resolution
F20	Multi-agent visibility: which agents hold suspect claims	P2	Dashboard lists agents per affected claim

Explicit exclusions: no automatic discovery of every dependency (only observed reads and declared claims), no semantic proof of correctness, no blocking by default.

Core model and data schema

Everything the product knows is a node in an evidence graph with a revision-stamped state. Observations come from tool logs and are trusted most. Claims and decisions come from agent text and are trusted least until backed by evidence.

Node types

Type	Meaning	Source	Example
Observation	A fact the agent directly saw	Capture hook	Read `app/auth.py` at commit abc123, hash X
Claim	A belief derived from observations	Agent or extractor	Authentication uses JWT
Decision	A choice justified by claims	Agent or extractor	Build refresh-token middleware
Action	A proposed or executed change	Capture hook	Edit `app/middleware.py`

Claim states

State	Meaning	Becomes this when
valid	Evidence matches current code under policy	All evidence hashes match, or an independent source still holds
suspect	A relevant source changed; not yet re-checked	Any evidence file changed, or an upstream node is suspect
invalid	New evidence contradicts it	A verification check fails or a rule detects a contradiction
unknown	Not enough evidence to say	No evidence, or capture was incomplete

Transitions are only: valid to suspect on change, suspect to valid or invalid on verification, any to unknown on lost evidence. No state is set by an LLM alone.

Schema (PostgreSQL in cloud; same shape in SQLite locally)

```
CREATE TABLE repos (
  id UUID PRIMARY KEY, provider TEXT, full_name TEXT, default_branch TEXT);

CREATE TABLE sessions (
  id UUID PRIMARY KEY, repo_id UUID REFERENCES repos,
```

```
  agent_kind TEXT, agent_id TEXT, base_commit TEXT,
  started_at TIMESTAMPTZ, ended_at TIMESTAMPTZ);

CREATE TABLE nodes (
  id UUID PRIMARY KEY, session_id UUID REFERENCES sessions,
  type TEXT CHECK (type IN ('observation','claim','decision','action')),
  text TEXT, state TEXT CHECK (state IN
('valid','suspect','invalid','unknown')),
  origin TEXT CHECK (origin IN ('tool_log','agent','llm_extracted')),
  created_at TIMESTAMPTZ, state_changed_at TIMESTAMPTZ);

CREATE TABLE evidence (
  id UUID PRIMARY KEY, node_id UUID REFERENCES nodes,
  repo_id UUID REFERENCES repos, commit_sha TEXT,
  path TEXT, content_sha256 TEXT, symbol TEXT NULL,
  observed_at TIMESTAMPTZ);

CREATE TABLE edges (
  from_node UUID REFERENCES nodes, to_node UUID REFERENCES nodes,
  kind TEXT CHECK (kind IN ('supports','derived_from','plans')),
  PRIMARY KEY (from_node, to_node, kind));

CREATE TABLE changes (
  id UUID PRIMARY KEY, repo_id UUID REFERENCES repos,
  before_sha TEXT, after_sha TEXT, path TEXT, change_kind TEXT,
  new_sha256 TEXT, detected_at TIMESTAMPTZ);

CREATE TABLE decisions (
  id UUID PRIMARY KEY, action_node UUID REFERENCES nodes,
  verdict TEXT CHECK (verdict IN ('allow','verify','block','unresolved')),
  reason TEXT, required_checks JSONB, policy_version TEXT,
  bound_base_sha TEXT, expires_at TIMESTAMPTZ, created_at TIMESTAMPTZ);

CREATE TABLE verifications (
  id UUID PRIMARY KEY, node_id UUID REFERENCES nodes,
  check_kind TEXT, result TEXT CHECK (result IN
('pass','fail','inconclusive')),
  detail JSONB, ran_at TIMESTAMPTZ);

CREATE INDEX ON evidence (repo_id, path);
CREATE INDEX ON edges (to_node);
```

Why relational edges first: the graphs are small per session (hundreds of nodes), and a recursive query handles reachability. A dedicated graph database adds operations cost before any evidence it is needed.

Provenance rule: every node records its origin. A node whose origin is `llm_extracted` is never promoted to `valid` without matching evidence rows.

LLM strategy: what checks, what must not

Direct answer. Yes, an LLM does some checking, but it only proposes. A deterministic layer confirms its evidence and makes the final call. The rule: an LLM may add suspicion freely, but it may remove suspicion only with a cited line that the code verifies, and never alone on a high-risk action.

Where the LLM is used (four jobs)

Job	Input	Output (strict JSON)	Guardrail
1. Claim extraction	Agent notes, plan text, tool logs	Claims with evidence paths and dependencies	Claims lacking matching read-log evidence are stored as *unknown*; origin tagged `llm_extracted`
2. Change interpretation	Diff hunks for a changed file plus the claim text	`affects_claim: yes / no / unsure`, plus cited lines	Cited lines must exist verbatim in the diff; mismatch makes it *unsure*
3. Semantic re-verification	Claim plus current file excerpt	`still_true / false / unsure`, plus cited lines	Verified citation required; high-risk actions still need a test or human
4. Explanation	Verified evidence chain	Plain-language reason for the PR comment	Text only; cannot change any state

Where the LLM must never decide

- Whether a file changed (hashes).
- Which nodes are reachable from a change (graph traversal).
- Whether an action is allowed, verified or blocked (policy rules).
- Whether a claim becomes *valid* again after a high-risk change (needs a passing test, a re-read matching the current hash, or human approval).

Why not an "LLM judge" for the gate: the same family of models that wrote the stale assumption would grade it. Its errors correlate with the agent's, it is non-deterministic across runs, and a blocked action must be explainable and replayable. Hashes and rules

are.

Model choice (recommendation, to be settled by the benchmark)

Job	First choice	Why	Fallback
Extraction and explanation	Claude Haiku 5.5	High volume, simple structured output, lowest latency and cost	Any small model with JSON schema support
Change interpretation and re-verification	Claude Sonnet 5.5	Better at reading code diffs; fewer misses matter more here	Larger model only if benchmark recall demands it
Offline or sensitive repos	Self-hosted open-weight model via a local runtime	Code never leaves the machine	Run jobs 1 and 4 only; leave 2 and 3 to deterministic checks

These are starting hypotheses, not measured. Build a provider adapter from day one so any model can be swapped, and choose models by benchmark results on your own scenarios, not by general leaderboards.

Controls

- Structured output with schema validation; retry once on invalid JSON; then mark *unsure*.
- Temperature 0 for jobs 1 to 3; log model name and prompt version on every output.
- Cache by content hash: the same file excerpt and claim never triggers a second call.
- Call the LLM only for nodes already flagged *suspect* or *unknown*, never on every commit.
- Per-repo budget with hard cap; when exceeded, return *unresolved* instead of guessing.

Cost model: monthly LLM cost per repo = (flagged nodes per month × average tokens per check × price per token) + extraction calls. All inputs are unknown until pilots; track tokens per resolved flag from day one.

Privacy: send only the minimum excerpt needed, strip secrets by pattern before any call, and make the LLM step optional. The product must function in a deterministic-only mode.

Core algorithms

1. Change detection (deterministic)

1. Resolve the base commit the agent started from and the current head of the target branch.
2. Run `git diff --name-status -M base..head` to get added, modified, deleted and renamed paths.
3. For each path the agent read, compare the recorded SHA-256 against the blob at head. Equal hash means unchanged, even if the commit touched the file.
4. Optional normalization: ignore whitespace-only and comment-only changes by parsing with Tree-sitter and comparing syntax trees (policy flag, off by default in v0).
5. Output a change set C of (path, change kind, new hash).

2. Impact propagation

1. Seed: every evidence row whose path is in C marks its node *suspect*.
2. Traverse `supports`, `derived_from` and `plans` edges forward with a recursive query; every reached node becomes *suspect*, with a reason that names the originating change.
3. Independent evidence check: if a reached node has another supporting source that is unchanged and sufficient under policy, keep it *valid* and note the unchanged source.
4. Stop at actions already executed; record them as "executed on suspect basis" for review.

```
I(C) = \operatorname{Reachable}(C, G)
```

Reachable means potentially affected. Only verification can promote a node to *valid* or *invalid*.

3. Policy decision (pure function)

Inputs: action, target paths, risk level, node states, coverage, policy version. The same inputs always produce the same output.

Condition	Verdict
All supporting nodes valid, coverage meets policy	allow
Any supporting node suspect, risk low or medium	verify, with the cheapest resolving checks
Any supporting node suspect, risk high	block until checks pass
Any supporting node invalid	block
Evidence missing or capture incomplete, risk high	unresolved, request human review

Condition	Verdict
Evidence missing, risk low	allow with a logged warning
Verification budget exhausted	unresolved, never a false allow

Risk comes from the policy file by path pattern (for example, auth, migrations and billing as high) and defaults to medium.

4. Verification planner

Rank checks by cost and power: re-read the file and compare hash (cheapest), compare the changed symbols (Tree-sitter), run mapped tests in a sandbox, LLM re-verification with a verified citation, then human review. For each suspect node, emit the cheapest check likely to resolve it, and stop when the budget is reached.

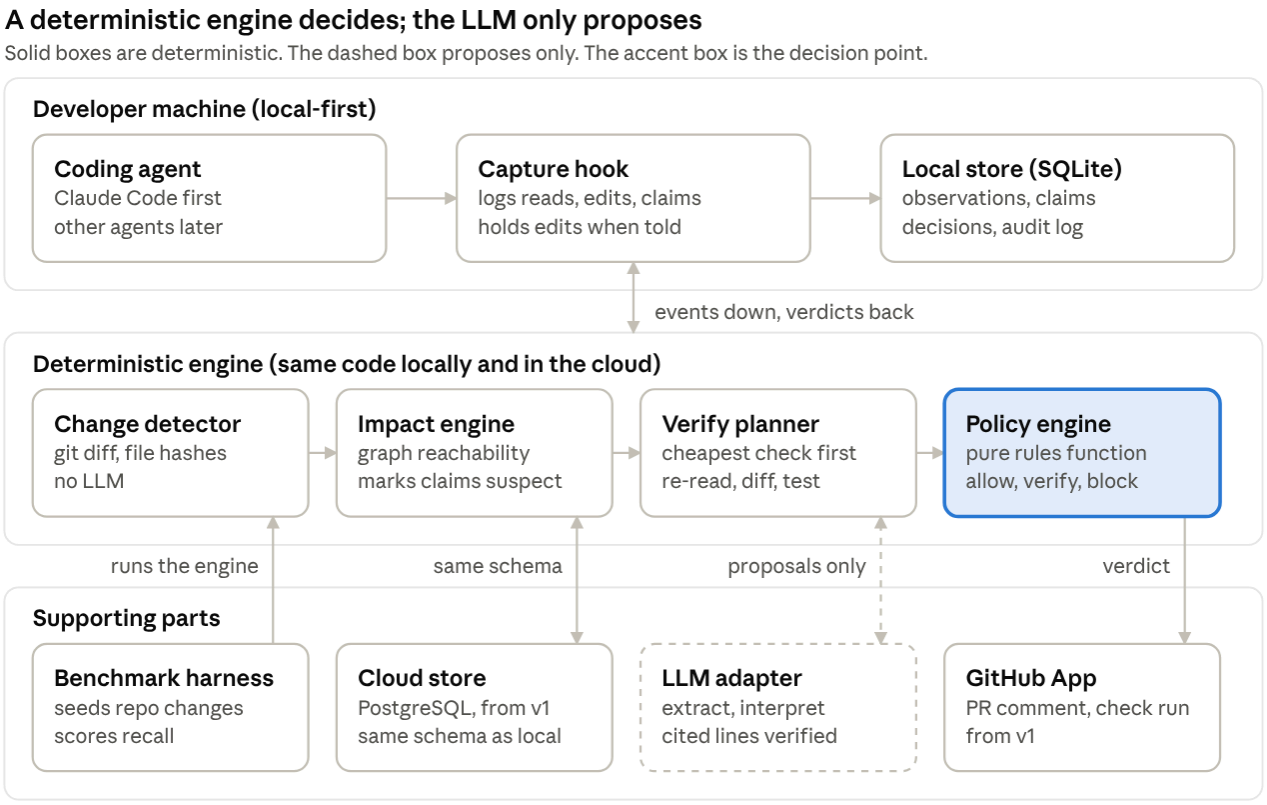
5. The merge race

The repository can change after a check passes but before merge. Every allow stores `bound_base_sha` and an expiry. At the execution boundary (merge or edit), recompute the head SHA and the evidence hashes. If either differs, discard the allow and re-run analysis. This is the one place where a weaker design silently fails.

6. Coverage and incomplete graphs

Coverage is the share of the action's relevant files that appear in the agent's read log or declared evidence. When coverage is below the policy threshold, the verdict degrades toward *unresolved* instead of pretending to know. Hidden dependencies (config, reflection, external services) are not detected in v0, and the PR comment states that limit.

System architecture



architecture · 3 zones, LLM proposes only

Events flow from the agent through the hook into deterministic parts, and the verdict returns to the hook. The LLM adapter is the only non-deterministic part and can only propose; the policy engine decides.

Tech stack: choices and rejected alternatives

The guiding rule is one language, few moving parts, and no infrastructure that a team of 3 has to operate before the benchmark proves demand. Choices are my recommendations; none have been load-tested.

Layer	Choice	Why it wins here	Rejected, and why
Core language	Python 3.12	Fastest for a 3-person team; the agent and AI ecosystem is Python-first; one language for engine, CLI, API and GitHub App	**TypeScript:** better for the dashboard later, split codebase now. **Rust:** fastest hooks, but slower to build and hire for; revisit if hook latency hurts. **Go:** fine single binary, weaker AI libraries
Packaging	uv and pipx-style install	One command install, isolated environment	**Docker-only:** heavy for a hook that runs on every edit
Git access	`git` CLI via subprocess; pygit2 later if needed	Exact behavior developers expect; renames and blob IDs handled correctly	**GitPython:** slow and leaky on large repos. **Libgit2 first:** extra build friction
Change fingerprint	Git blob object ID plus SHA-256 of captured content	Blob IDs are free to read at any commit; SHA-256 covers uncommitted local reads	**Timestamps:** unreliable, the failure the product exists to fix
Code structure	Tree-sitter via its Python bindings (pre-built wheels exist)	Language-agnostic syntax trees; symbol-level diffs without a language server	**LSP servers:** heavy, per-language setup. **Regex:** breaks on real code. **Full static analysis:** years of work
Local store	SQLite	Zero install, single file, same SQL as the cloud	**Postgres locally:** too heavy for a CLI. **JSON files:** no queries, no recursion

Layer	Choice	Why it wins here	Rejected, and why
Cloud store	PostgreSQL (managed)	Relational edges and recursive CTEs cover graph reachability; one database to run	**Neo4j or other graph DBs:** extra ops cost with no proven need at hundreds of nodes per session. **Vector DBs:** this product does not need semantic retrieval. **DuckDB:** analytics, not transactions
Background jobs	Postgres-backed queue first	Removes Redis and a broker from v1	**Redis plus Celery:** reasonable at scale, but two more services for v1
API	FastAPI with Pydantic	Typed schemas double as the JSON contracts for SDK and CLI	**Django:** heavier than needed. **Flask:** no built-in validation
GitHub integration	A GitHub App with `checks:write` and PR comment permissions	Check runs can only be created by GitHub Apps, not by OAuth apps (GitHub docs)	**Webhook plus personal token:** cannot create check runs, worse security. **GitHub Action only:** simple, but no persistent state across runs
Agent integration	Claude Code hooks first (`PreToolUse` can block a call)	Real enforcement point, big user base (docs)	**MCP-only integration:** advisory, the agent can ignore it. **Forking an agent:** no
LLM access	Anthropic SDK behind a provider adapter	Model swap without rewriting; matches the benchmarked default	**LangChain-style frameworks:** abstraction overhead, hard to test. **Single-vendor lock-in:** blocks the offline mode
Schemas and policy	Pydantic models; YAML policy file validated on load	One source of truth for validation and JSON Schema export	**Free-form JSON:** silent drift

Layer	Choice	Why it wins here	Rejected, and why
Testing	pytest, Hypothesis for property tests on the graph, Dockerized fixture repos	Property tests catch propagation edge cases	**Manual QA:** the product's claim is correctness
CI and release	GitHub Actions	Same platform as the product	Other CI: no benefit here
Hosting (v1)	One container platform plus managed Postgres	Fewest things to operate	**Kubernetes:** premature. **Self-managed VMs:** more on-call burden
Dashboard (P2)	Next.js and React, built only after paid pilots	Standard, hireable	**Building it now:** a dashboard is not the validation target
Telemetry	Structured logs now, OpenTelemetry export later	Avoid early lock-in; check GenAI conventions before relying on them	**Custom metrics system:** wasted effort

Decisions that are easy to reverse: hosting, queue, dashboard framework. **Decisions that are costly to reverse:** the evidence and state schema, the policy contract, and the hook interface. Spend design time on those three.

Interfaces

All names, flags and fields below are proposed designs, not existing software.

1. CLI

```
verix init                      # create .verix.yml, install hooks, create
local DB
verix session start --agent claude-code
verix claim add "Auth uses JWT" --evidence app/auth.py
verix analyze --base abc123 --head HEAD
verix check --action edit --path app/middleware.py --risk high
verix verify <claim-id>         # run the suggested check
verix report --json
verix bench run --scenarios 30   # benchmark harness
```

`check` exit codes: 0 allow, 10 verify, 20 block, 30 unresolved. A hook maps these to Claude Code behavior.

2. Claude Code hook contract

Hook event	What Verix does
SessionStart	Records base commit; opens a session; prints a short brief of suspect claims from prior sessions
PostToolUse (Read)	Logs path, commit, content hash as an observation
PreToolUse (Edit, Write)	Runs `check`; on block, exits with code 2 so the call does not run, and returns the reason to the agent
PreCompact	Persists the current claim and read log so a resumed session can be re-checked
Stop	Writes a session summary and the final decision log

The hook must finish within a latency budget (target 300 ms median, to be measured) and fail open in advisory mode.

3. Policy file `.verix.yml`

```
version: 1
mode: advisory            # advisory | blocking
risk:
  high:   ["app/auth/**", "migrations/**", "billing/**"]
  medium: ["app/**"]
  low:    ["docs/**", "tests/**"]
coverage:
  minimum_read_ratio: 0.6
normalize:
  ignore_whitespace: false
llm:
  enabled: false          # deterministic-only by default
  budget_usd_per_month: 20
checks:
  tests:
    auth: "pytest tests/test_auth.py"
```

4. REST API (v1, cloud)

Method and path	Purpose
POST /v1/sessions	Open a session for a repo and base commit
POST /v1/observations	Batch-upload read and write logs

Method and path	Purpose
POST /v1/claims	Create a claim with evidence and parents
POST /v1/changes/analyze	Submit before and after commits; returns affected nodes and checks
POST /v1/actions/evaluate	Return allow, verify, block or unresolved with reasons
POST /v1/verifications	Record the result of a check
GET /v1/decisions/{id}	Replay a decision with its inputs

Example response of `actions/evaluate`:

```
{
  "decision_id": "dec_019",
  "verdict": "verify",
  "reason": "app/auth.py changed since claim_auth_017 was observed",
  "affected": ["claim_auth_017", "decision_mw_004"],
  "required_checks": [{"kind": "diff", "path": "app/auth.py"}, {"kind":
"test", "cmd": "pytest tests/test_auth.py"}],
  "bound_base_sha": "def456",
  "expires_at": "2026-11-03T12:00:00Z",
  "policy_version": "1"
}
```

5. GitHub App behavior

- Permissions: contents read, pull requests write, checks write, metadata read.
- Events: `pull_request` opened and synchronize, `push` to the base branch.
- On a PR: look up the agent session by branch or commit trailer, run analysis, post or update one comment, and set a check run to neutral (advisory), success or failure (blocking).
- Agent association: the hook adds a `Verix-Session: <id>` trailer to agent commits; PRs without a session get no comment.
- Idempotent: re-running on the same head and base produces the same result.

6. Python SDK (for agent-tool builders)

```
from verix import Verix

guard = Verix(project="backend-service")
session = guard.sessions.start(base_commit="abc123")
claim = guard.claims.create(session, text="Auth uses JWT", evidence=
["app/auth.py"])
decision = guard.actions.evaluate(session, action="edit", paths=
["app/middleware.py"], risk="high")
if decision.verdict != "allow":
    handle(decision.required_checks)
```

Usage modes

Users reach Verix through several surfaces that share one engine and one record format. Capture happens on the developer machine; PR surfaces read a small receipt that travels with the pull request.

Surface	Who uses it	When it runs	Can it block?	Needs	Version
Claude Code hooks	Agent operator	During the session	Yes, via PreToolUse	Local install, hooks added by `init`	v0
CLI	Anyone	On demand	Exit codes only	Local install	v0
CI step (GitHub Action)	Team	On every PR push	Yes, by failing a check	Session receipt in the branch or PR	v1
Codex CLI	Agent operator	During the session	Yes if its hooks feature is enabled; otherwise advisory	Hook adapter; verify against OpenAI's current hooks docs	v1
MCP server plus `AGENTS.md`	Operators of agents without hooks	When the agent calls the tool	No, advisory only	Local install	v1
GitHub App	Team or organization	On PR events	Yes, via a check run	Receipt or uploaded metadata, hosting	v2
Python SDK	Agent-tool builders	Inside their own agent	Their choice	`pip install`	v2

Codex hook support is based on third-party tooling and starter kits, not on OpenAI's documentation read directly. Treat parity with Claude Code as unconfirmed.

The session receipt. CI and the GitHub App cannot see what an agent read locally. The hook writes a receipt, and the PR surface compares it with the current base branch.

```
{
  "session_id": "ses_042",
  "agent": "claude-code",
  "base_commit": "abc123",
  "reads": [{"path": "app/auth.py", "blob": "9f2c...", "sha256": "a41b..."}],
  "claims": [{"id": "c1", "text": "Auth uses JWT", "evidence":
["app/auth.py"]}],
  "actions": [{"kind": "edit", "path": "app/middleware.py"}]
}
```

- Stored at `.verix/receipts/<session_id>.json` on the branch, or attached to the PR as an artifact (to be decided in prototype).
- Contains paths, hashes and claim text, never file bodies.
- A receipt is self-reported by the developer's machine. An agent or user can omit reads, so a missing receipt produces *unresolved*, not a pass.

Typical flow

1. `verix init` installs hooks and the policy file.
2. The agent works; the hook writes the receipt and may hold risky edits.
3. The agent opens a PR; the receipt travels with it.
4. The CI step or App compares receipt hashes with the base branch and posts one comment and one check.
5. The agent or a human re-verifies the listed items; the next push updates the comment.

Order of delivery: Claude Code hooks and CLI, then the CI step, then Codex, then the GitHub App. The CI step proves the PR value with no server to host.

Open questions: whether committing receipts to the repository is acceptable to teams, how large receipts get on long sessions, and whether other agents expose enough to produce a faithful read log.

Experience specs

Design rule: every flag names a file, a commit, and the next action. No generic warnings, no walls of text, and silence when nothing is wrong.

PR comment (v1)

```
### Verix: 2 of 5 assumptions need re-checking

| Status | Assumption | Why |
| --- | --- | --- |
| Suspect | Auth uses JWT | `app/auth.py` changed after the agent read it
(abc123 to def456) |
| Suspect | Refresh middleware is unused | Depends on the claim above |
| Valid | DB schema has a users table | `db/schema.sql` unchanged |

**To resolve:** diff `app/auth.py`, run `pytest tests/test_auth.py`
**Coverage:** the agent read 8 of 11 relevant files. Dependencies outside
these files are not checked.

Mode: advisory. Re-run: push a new commit.
```

Rules: one comment per PR, edited in place, collapsed details for long lists, a plain statement of limits, and a link to the evidence chain once the dashboard exists.

CLI output (v0)

```
$ verix check --action edit --path app/middleware.py --risk high
VERIFY  app/middleware.py (risk: high)
  claim_auth_017 "Auth uses JWT" is suspect
    app/auth.py changed since it was read (abc123 -> def456)
  next: verix verify claim_auth_017
        or run: pytest tests/test_auth.py
```

What the agent sees on a block (Claude Code)

A short, actionable message: which claim is stale, which file changed, and the exact command to resolve it. The message must not tell the agent to bypass the guard.

Dashboard screens (P2, build only after paid pilots)

1. **Repo overview:** active sessions, suspect claims, held actions, resolved checks in the last 24 hours.
2. **Flag detail:** the evidence chain, source change, affected claim, dependent decision, proposed action, required check, and outcome. This chain is the heart of the product.
3. **Policy editor:** edit risk patterns and mode with a dry-run showing what past PRs would have flagged.
4. **Benchmark view:** recall, precision and false-block rate over time for the customer's own repos.

UX principles

- Advisory by default; blocking is an explicit opt-in.

- Report limits honestly: coverage, unknown dependencies, unresolved results.
- Fail open when Verix itself breaks, and say so in the PR check.
- Never show raw LLM text as a verdict; show cited evidence.

Non-functional requirements

Targets marked *(validate)* are starting hypotheses, not measured numbers.

Area	Requirement	Target
Latency	Hook check on a protected edit	Median 300 ms, p95 1 s *(validate)*
Latency	Analysis of a PR with up to 500 changed files	Under 5 s without LLM calls *(validate)*
Determinism	Same inputs, same verdict	100%; replayable from the audit log
Availability	Verix failure must not stop developers	Fail open in advisory mode; surface a failure notice
Accuracy	Invalidation recall on the seeded benchmark	80% or higher
Accuracy	False-block rate on valid actions	Under 10%
Cost	LLM spend	Per-repo monthly cap; deterministic-only mode always available
Scale (v1)	Concurrent sessions per repo	Tens; design for hundreds *(validate)*
Portability	OS support	macOS and Linux first; Windows after demand

Security

- Local-first: by default only hashes, paths, claim text and decisions leave the machine. Source code never uploads unless a customer opts in per repo.
- Strip secrets by pattern from any text before an LLM call or upload; never log file contents.
- GitHub App uses least-privilege permissions and short-lived installation tokens; no stored user passwords in v1.
- Encrypt data at rest and in transit; per-tenant isolation in the database (row-level security).
- Hooks are not a security boundary. State this plainly in docs: strict enforcement needs a trusted execution gateway, which is out of scope.

- Treat all agent text, repository files and LLM output as untrusted input; never execute commands taken from them without the policy's allow-list.

Privacy and data retention

- Configurable retention (default to be set with design partners); delete a repo's data on request.
- Clear table of what is stored: node text, evidence references, decisions, no source bodies by default.
- LLM calls: documented which excerpts are sent, to which provider, and how to turn it off.

Reliability

- Idempotent webhooks and analysis jobs; retries with backoff.
- A bound base SHA on every allow, rechecked at the execution boundary.
- Versioned schemas and policy files with migrations; old audit logs stay replayable.

Compliance: none claimed in v0. Enterprise needs (SOC 2, data residency, self-hosting) come only if customers ask and pay.

Maintainability: one repository, one language, typed code, 80% coverage on the engine core, and property tests for graph propagation.

Evaluation spec

The benchmark is both the quality gate and the launch asset. Write the protocol before running any agent, and publish the scenarios so others can reproduce results.

Design. A scripted run: an agent reads a repository and writes a plan, then pauses. The harness applies a seeded change, resumes the agent, and records whether it acts on stale assumptions. Ground-truth labels (which claims are truly affected) are written by a person for each scenario, independent of the system under test.

Scenario families (30 in v0)

Family	Example	Count (target)
Renamed or removed symbol	Function the agent plans to call is renamed	6
Behavior change in a dependency file	Auth moves from JWT to sessions	6
Schema or config change	Column dropped, env var renamed	5

Family	Example	Count (target)
Dependency version bump	Incompatible API in a library	4
Irrelevant change (must stay silent)	Docs or unrelated module edited	5
Whitespace or refactor only	Formatting change, same behavior	2
Hidden dependency (not in read log)	Runtime config affects behavior	2

Conditions compared

1. No guard (baseline agent).
2. File-hash flagging only (the projectmem-style approach, re-implemented or run directly).
3. Verix deterministic mode.
4. Verix with LLM jobs enabled.

Metrics

Metric	Definition
Invalidation recall	Affected assumptions flagged divided by all truly affected
Precision	Flagged assumptions that truly needed re-checking divided by all flagged
Unsafe-action rate	Actions violating a defined precondition, divided by all actions
False-block rate	Valid actions blocked, divided by valid actions
Task completion	Valid tasks still completed
Recovery success	Agent replans correctly after a flag
Overhead	Added latency, tokens and dollars per task

```
\text{Invalidation Recall} = \frac{TP}{TP + FN}
```

Run protocol. At least 5 runs per scenario per condition because agent behavior varies; report mean and range; pin model versions and prompts; publish raw logs.

Pass thresholds for v0 (decision rules)

- Recall 80% or higher and false-block rate under 10% in condition 3.
- Clear gap over condition 2 on transitive scenarios. If condition 2 catches 75% or more of what condition 3 catches, drop the graph and pivot.
- LLM jobs (condition 4) must add recall or cut false blocks enough to justify their cost, or they stay off by default.

Do not advertise a percentage reduction in failures until these experiments produce it. Report what the benchmark measures, and what it does not (real-world frequency of the problem).

Build plan

Build in an order that kills the idea cheaply if it is wrong. Each phase has a gate; do not start the next phase if the gate fails. Week ranges are targets for a focused team of 3 alongside other commitments, not delivery promises.

Phases and gates

1. **Weeks 1 to 2, foundations.** Repo, CI, schema, fixture repos, benchmark scenario list, interview script, and 5 interviews. *Gate:* the 30 scenarios and their ground-truth labels are written and reviewed.
2. **Weeks 3 to 5, deterministic engine.** Change detection, evidence capture, impact propagation, policy function, local SQLite store, CLI. *Gate:* engine passes all fixture tests, including the property tests for propagation.
3. **Weeks 4 to 6, Claude Code hook.** SessionStart, Read logging, PreToolUse check, PreCompact persistence. *Gate:* a recorded demo of two agents on one repo where the second is held until it re-reads.
4. **Weeks 6 to 8, benchmark runs.** Run all four conditions, 5 runs per scenario. Publish results and code. *Gate:* the pass thresholds in the evaluation spec. Failing them triggers a pivot decision.
5. **Weeks 8 to 10, CI step then GitHub App.** GitHub Action first (PR comment and check run, no server, reads the session receipt), then the GitHub App once partners ask for history and organization-wide policy. *Gate:* 3 design partners installed on real repos.
6. **Weeks 10 to 12, pilots and pricing.** Measure flags acted on and rework avoided; test willingness to pay. *Gate:* at least one partner pays $500 or more, or commits in writing.
7. **After the gate:** LLM jobs, claim extraction, hosted history, dashboard, additional agent integrations. None of these start before the previous gate passes.

Team split (roles, not names; adjust to your team's skills)

Role	Owns	Early deliverables
Engine lead	Schema, change detection, graph propagation, policy function	Phases 2 and 4
Integration lead	Claude Code hook, CLI, later GitHub App and API	Phases 3 and 5
Research and customer lead	Benchmark protocol and labels, interviews, design partners, pricing tests, write-up	Phases 1, 4 and 6

The research lead also drives the paper, which keeps its value if the product gate fails.

Working agreements

- Weekly demo against the benchmark; the recall and false-block numbers are the team's scoreboard.
- No feature enters the build unless it maps to a functional requirement with an acceptance criterion.
- Time-box agency work so a 3-person team keeps at least 50% of capacity on this (an assumption; set your own).

Definition of done for v0: all P0 requirements pass acceptance tests, the benchmark meets thresholds, install takes under 10 minutes on a clean machine, and a stranger can reproduce the results from the repository.

Risks, open questions and acceptance checklist

Technical and product risks (each paired with its test)

Risk	Likelihood	Impact	Mitigation	Test
Becomes a feature of a bigger platform (existential)	High	High	Benchmark lead; depth of policy and evidence chain	Do partners pay beyond free tools?
Read log misses what the agent actually relied on	High	Medium	Report coverage; degrade to unresolved	Hidden-dependency scenarios
Hook latency annoys developers	Medium	Medium	Cache; advisory fail-open; measure p95	Latency test on a 5,000-file repo

Risk	Likelihood	Impact	Mitigation	Test
False blocks cause the guard to be disabled	Medium	High	Advisory default; precision tuning	False-block rate and bypass rate in pilots
LLM errors erode trust	Medium	Medium	Cited-line verification; deterministic mode	Citation-match rate on the benchmark
Agent hook APIs change	Medium	Medium	Thin adapter layer; version pinning	Contract tests against hook payloads
Private code leaks through telemetry or LLM calls	Low	Very high	Local-first; secret stripping; opt-in uploads	Security review before any pilot
Team capacity is split with the agency	High	Medium	Time-box; one narrow release	Weekly scoreboard

Open questions (answer before or during v0)

1. Do agents expose enough in their logs to build a faithful read record outside Claude Code?
2. How should a claim be scoped so it is neither too broad (everything suspect) nor too narrow (nothing links)?
3. Should the PR guard act on agent identity, or on any PR that shares a session trailer?
4. What is the right default retention period, and is there a legal need for it in target customers?
5. Does symbol-level diffing meaningfully beat file-level hashing on real repos, or only on seeded ones?
6. Which risk-level defaults satisfy most teams without custom policy files?
7. What does a team actually pay for: fewer reworked PRs, review time saved, or audit evidence?

Acceptance checklist for v0

- [ ] All P0 functional requirements (F1 to F6, F8, F10 to F12, F15, F17, F18) pass their acceptance criteria
- [ ] Deterministic mode works with no network access
- [ ] Benchmark of 30 scenarios runs with one command and meets the pass thresholds
- [ ] Hook median latency measured and documented
- [ ] Policy file validation errors are line-level and readable

- [ ] Audit log replays any decision identically
- [ ] Install in under 10 minutes verified on a clean macOS and a clean Linux machine
- [ ] Security review: no source code or secrets leave the machine by default
- [ ] README states limits: not a security boundary, no proof of correctness, incomplete dependency view
- [ ] 20 interviews done and thresholds from the strategy document evaluated

What this PRD does not settle: demand and willingness to pay, which only interviews and paid pilots can answer; the real-world frequency of stale-PR incidents; and whether the platform vendors will ship this natively.

Sources checked for this document: GitHub check runs API · Claude Code hooks · py-tree-sitter bindings. Competitor and market evidence is in the first document.